

Advome Backend API Specification

Overview

This document defines the backend API architecture for the Advome platform. The backend connects the frontend HTML/CSS/JS interface to the Kimi AI engine, database, and payment gateway.

Technology Stack Recommendation

- **Backend:** Node.js with Express (or Python with FastAPI)
- **Database:** PostgreSQL (structured data) + AWS S3 (document storage)
- **AI Engine:** Kimi API (Moonshot AI) via REST API
- **Payment:** PayFast (South Africa) or Stripe
- **Auth:** JWT (JSON Web Tokens) with bcrypt password hashing
- **Hosting:** Heroku / Render / AWS / DigitalOcean

Base URL

```
Production: https://api.advome.co.za/v1
Development: http://localhost:3000/v1
```

Authentication

All endpoints except `/auth/register` and `/auth/login` require a Bearer token:

```
Authorization: Bearer <jwt_token>
```

Endpoints

1. Authentication

POST `/auth/register` Register a new user.

```
{
  "email": "user@example.com",
  "password": "secure_password",
  "full_name": "John Doe",
  "phone": "+27 82 123 4567",
  "plan": "self" // or "attorney"
}
```

Response:

```
{
  "success": true,
  "token": "jwt_token_here",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "plan": "self",
    "created_at": "2026-06-13T22:00:00Z"
  }
}
```

```
}
```

POST /auth/login Login existing user.

```
{
  "email": "user@example.com",
  "password": "secure_password"
}
```

Response:

```
{
  "success": true,
  "token": "jwt_token_here",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "plan": "self"
  }
}
```

2. Document Detection (Kimi Integration)

POST /documents/detect Upload and analyze a legal document using Kimi AI.

Content-Type: multipart/form-data

Fields:

- file (required): PDF, DOC, DOCX, PNG, JPG
- matter_type (optional): hint about matter type

Response:

```
{
  "success": true,
  "detection": {
    "document_type": "CCMA Arbitration Award",
    "confidence": "High",
    "detected_stage": "Award Received",
    "matter_category": "labour",
    "next_steps": [
      "Record service date",
      "Read award carefully",
      "Decide: accept or challenge within 6 weeks"
    ],
  },
  "extracted_data": {
    "award_date": "2026-05-15",
    "commissioner_name": "Commissioner Smith",
    "outcome_summary": "Dismissal upheld"
  },
  "recommended_workflow": "award.html"
}
```

```
}  
}
```

Kimi Prompt Used:

```
You are a South African legal document analyzer. Analyze the following document and return ONLY a JSON  
↪ object with these fields:  
- document_type: string  
- confidence: "High" | "Medium" | "Low"  
- detected_stage: string  
- matter_category: "labour" | "divorce" | "estates" | "civil" | "unknown"  
- next_steps: array of strings (3-5 steps)  
- extracted_data: object with key facts  
- recommended_workflow: string (HTML filename)  
  
Document text: [extracted_text]
```

3. Template Generation (Kimi Integration)

POST /templates/generate Generate a legal document template using Kimi AI.

```
{  
  "template_type": "notice_of_motion",  
  "matter_id": "uuid",  
  "user_inputs": {  
    "applicant_name": "John Doe",  
    "respondent_name": "Company XYZ",  
    "relief_sought": "setaside",  
    "court_division": "johannesburg",  
    "grounds": ["misconduct", "irregularity"]  
  }  
}
```

Response:

```
{  
  "success": true,  
  "template": {  
    "id": "uuid",  
    "type": "notice_of_motion",  
    "content": "IN THE LABOUR COURT OF SOUTH AFRICA...",  
    "generated_at": "2026-06-13T22:00:00Z",  
    "disclaimer": "This is a template generated by Advome...",  
    "word_count": 450  
  }  
}
```

Kimi Prompt Used:

```
You are a South African legal document drafter. Generate a [TEMPLATE_TYPE] based on the following user  
↪ inputs.  
Use proper South African legal formatting and language. Include all standard sections.
```

Return ONLY the document text with a disclaimer footer.

User inputs: [JSON_inputs]

4. Matter Management

GET /matters List all matters for the authenticated user. Response:

```
{
  "success": true,
  "matters": [
    {
      "id": "uuid",
      "title": "Mmoledi v Mmoledi",
      "type": "divorce",
      "current_stage": "pleadings",
      "status": "active",
      "readiness_score": 55,
      "urgency": "attention",
      "created_at": "2026-06-01T10:00:00Z",
      "updated_at": "2026-06-13T15:00:00Z"
    }
  ]
}
```

POST /matters Create a new matter.

```
{
  "title": "Mmoledi v Mmoledi",
  "type": "divorce",
  "description": "Divorce proceedings",
  "parties": {
    "plaintiff": "John Mmoledi",
    "defendant": "Jane Mmoledi"
  }
}
```

GET /matters/:id Get specific matter details.

PUT /matters/:id/stage Update matter stage.

```
{
  "stage": "settlement",
  "notes": "Settlement negotiations ongoing"
}
```

5. Document Storage

POST /documents/upload Upload a document to matter file.

Content-Type: multipart/form-data

```
Fields:
- file (required)
- matter_id (required)
- document_type (optional): "summons", "plea", "award", etc.
```

GET /documents/:matter_id List all documents for a matter.

GET /documents/:id/download Download a specific document.

6. Fee Tracking

GET /fees/:matter_id Get fee breakdown for a matter. Response:

```
{
  "success": true,
  "total": 7500,
  "currency": "ZAR",
  "breakdown": [
    {
      "category": "filing",
      "amount": 1500,
      "status": "paid",
      "date": "2026-06-01"
    },
    {
      "category": "transcription",
      "amount": 3500,
      "status": "pending",
      "estimated": true
    }
  ]
}
```

POST /fees/:matter_id Add or update a fee item.

```
{
  "category": "filing",
  "amount": 1500,
  "status": "paid",
  "date": "2026-06-01",
  "notes": "High Court filing fee"
}
```

7. Attorney Portal

GET /attorneys List available attorneys (for client search).

POST /attorneys/connect Request connection with an attorney.

```
{
  "attorney_id": "uuid",
}
```

```
{
  "matter_id": "uuid",
  "message": "I need help with my CCMA review"
}
```

GET /attorneys/clients For attorney users: list connected clients.

POST /attorneys/share Share matter with attorney (client initiates).

```
{
  "matter_id": "uuid",
  "attorney_id": "uuid",
  "documents": ["doc_id_1", "doc_id_2"],
  "permissions": ["read", "comment"]
}
```

8. Billing & Time Tracking (Attorney Only)

POST /billing/time-entry Log time entry.

```
{
  "matter_id": "uuid",
  "date": "2026-06-13",
  "hours": 2.5,
  "rate": 800,
  "description": "Drafting plea",
  "category": "drafting"
}
```

GET /billing/summary Get monthly summary.

9. Legislation & Precedent

GET /legislation/search Search legislation database.

```
GET /legislation/search?q=section+145+LRA&category=labour
```

Response:

```
{
  "success": true,
  "results": [
    {
      "title": "Labour Relations Act 66 of 1995",
      "section": "145",
      "text": "A party may apply to the Labour Court for an order reviewing an arbitration award...",
      "citation": "LRA s145(1)",
      "relevance_score": 0.95
    }
  ]
}
```

Note: This can be powered by Kimi with a RAG (Retrieval-Augmented Generation) system or by maintaining a local legislation database.

10. Subscription & Payments

GET /subscription/status Check current subscription status.

POST /subscription/upgrade Upgrade plan.

```
{
  "plan": "attorney",
  "payment_method": "payfast"
}
```

POST /webhooks/payfast PayFast payment webhook handler.

Database Schema (PostgreSQL)

Users Table

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(255),
  phone VARCHAR(50),
  plan VARCHAR(20) DEFAULT 'self', -- self, attorney, enterprise
  firm_name VARCHAR(255),
  practice_number VARCHAR(50),
  province VARCHAR(50),
  practice_areas TEXT[],
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

Matters Table

```
CREATE TABLE matters (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  title VARCHAR(255) NOT NULL,
  type VARCHAR(50) NOT NULL, -- divorce, labour, estates, civil
  current_stage VARCHAR(100),
  status VARCHAR(20) DEFAULT 'active',
  readiness_score INTEGER DEFAULT 0,
  urgency VARCHAR(20) DEFAULT 'neutral',
  parties JSONB,
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

```
);
```

Documents Table

```
CREATE TABLE documents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  matter_id UUID REFERENCES matters(id),  
  user_id UUID REFERENCES users(id),  
  filename VARCHAR(255),  
  original_name VARCHAR(255),  
  file_size INTEGER,  
  mime_type VARCHAR(100),  
  s3_key VARCHAR(500),  
  document_type VARCHAR(100),  
  detected_stage VARCHAR(100),  
  ai_analysis JSONB,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Templates Table

```
CREATE TABLE templates (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id),  
  matter_id UUID REFERENCES matters(id),  
  template_type VARCHAR(100) NOT NULL,  
  content TEXT NOT NULL,  
  user_inputs JSONB,  
  ai_prompt TEXT,  
  word_count INTEGER,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Fees Table

```
CREATE TABLE fees (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  matter_id UUID REFERENCES matters(id),  
  category VARCHAR(100) NOT NULL,  
  amount DECIMAL(10,2) NOT NULL,  
  status VARCHAR(20) DEFAULT 'pending', -- paid, pending, estimated  
  is_estimated BOOLEAN DEFAULT false,  
  date DATE,  
  notes TEXT,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Attorney-Client Connections Table

```
CREATE TABLE attorney_connections (  

```



```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
attorney_id UUID REFERENCES users(id),
client_id UUID REFERENCES users(id),
matter_id UUID REFERENCES matters(id),
status VARCHAR(20) DEFAULT 'pending', -- pending, active, terminated
permissions JSONB DEFAULT '["read"]',
shared_documents UUID[],
created_at TIMESTAMP DEFAULT NOW()
);

```

Kimi API Integration

Configuration

```

const KIMI_API_KEY = process.env.KIMI_API_KEY;
const KIMI_BASE_URL = 'https://api.moonshot.cn/v1';

```

Document Detection Function

```

async function detectDocumentWithKimi(documentText) {
  const prompt = `You are a South African legal document analyzer. Analyze the following document and
  ⇨ return ONLY a JSON object with these fields:
  - document_type: string (e.g., "CCMA Arbitration Award", "Divorce Summons")
  - confidence: "High" | "Medium" | "Low"
  - detected_stage: string (e.g., "Award Received", "Pleadings")
  - matter_category: "labour" | "divorce" | "estates" | "civil" | "unknown"
  - next_steps: array of 3-5 strings
  - extracted_data: object with key facts (dates, names, amounts)
  - recommended_workflow: string (HTML filename like "award.html")

  Document text: ${documentText.substring(0, 8000)}`;

  const response = await fetch(`${KIMI_BASE_URL}/chat/completions`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${KIMI_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      model: 'kimi-latest',
      messages: [{ role: 'user', content: prompt }],
      temperature: 0.1,
      response_format: { type: 'json_object' }
    })
  });

  const data = await response.json();
  return JSON.parse(data.choices[0].message.content);
}

```

Template Generation Function

```
async function generateTemplateWithKimi(templateType, userInputs) {
  const prompts = {
    notice_of_motion: `Generate a South African Labour Court Notice of Motion for a Section 145 review.
    ↳ Use proper legal formatting. Include: parties, relief, and basis.`,
    founding_affidavit: `Generate a South African Founding Affidavit for a Labour Court review. Include
    ↳ deponent details, background facts, and grounds of review.`,
    settlement_agreement: `Generate a South African divorce settlement agreement. Include property
    ↳ division, maintenance, and children arrangements.`,
    parenting_plan: `Generate a South African parenting plan under the Children's Act. Include living
    ↳ arrangements, contact schedule, and decision-making.`,
    covering_letter: `Generate a covering letter for the Master's Office for estate administration.
    ↳ Include deceased details and document list.`
  };

  const prompt = `${prompts[templateType]}

  User inputs: ${JSON.stringify(userInputs)}

  Generate the complete document with proper South African legal formatting. Add a disclaimer footer.`;

  const response = await fetch(`${KIMI_BASE_URL}/chat/completions`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${KIMI_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      model: 'kimi-latest',
      messages: [{ role: 'user', content: prompt }],
      temperature: 0.3,
      max_tokens: 4000
    })
  });

  const data = await response.json();
  return data.choices[0].message.content;
}
```

Security Considerations

1. **API Keys:** Store in environment variables, never in frontend code
2. **File Uploads:** Scan for malware, limit file size (max 10MB), whitelist MIME types
3. **Authentication:** Use JWT with 24-hour expiry, refresh tokens for extended sessions
4. **Data Encryption:** Encrypt sensitive data at rest (AES-256), use TLS 1.3 in transit
5. **Rate Limiting:** 100 requests/minute per user, 10 document uploads/hour
6. **CORS:** Restrict to <https://advome.co.za> and <https://www.advome.co.za>
7. **POPIA Compliance:** Implement data subject access requests, right to deletion, breach notification within 72 hours

Error Handling

Standard error response format:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": {
      "email": "Email is required",
      "password": "Password must be at least 8 characters"
    }
  }
}
```

Common error codes:

- AUTH_ERROR — Invalid or expired token
- VALIDATION_ERROR — Invalid input data
- NOT_FOUND — Resource not found
- RATE_LIMIT — Too many requests
- PAYMENT_REQUIRED — Subscription expired
- AI_ERROR — Kimi API error
- FILE_ERROR — Invalid file type or size

Deployment Checklist

- Set up PostgreSQL database
- Configure S3 bucket for document storage
- Set up Kimi API key (environment variable)
- Configure PayFast merchant account
- Set up JWT secret key
- Configure CORS origins
- Set up SSL certificate
- Configure rate limiting
- Set up monitoring (Sentry, LogRocket)
- Test all endpoints with Postman
- Load test with 100 concurrent users
- POPIA compliance audit
- Penetration testing
- Documentation for frontend developers